Write Up

- 學號：U1224054
- 姓名：王淯霆

- repo：https://github.com/shellkz/dormitory-management

Schema Design Rationale

Entity and Relation

We can describe scheme with below sentence.

```
User stay at Room.
User request maintenance for Room.
```

Since User and Room are the beings take and got taken action. They are entities. On the other hand, Stay and RequestMaintenance are action themself. They are relations.

Cardinality

Based on requirement, I need to keep track of stay and maintenance history. So the scope extends to history-wide. Let's reconsider above schema description but this time with history-wide scope.

Original description

```
User stay at Room.
User request maintenance for Room.
```

Should become

```
Accross history, User can stay at multiple Room.
Accross history, Room can accommodate multiple User.
```

Stay is a N to M relation.

Use the same way let's reexamine RequestMaintenance

```
Accross history, User can request maintenance for multiple Room.
Accross history, Room can receive request maintenance from multiple User.
```

Now we can also conclude that RequestMaintenance is also a N to M relation.

That why I deicde make both of Stay and RequestMaintenance junction tables which use FK point to User and Room.

At this point, we can see overall Relational Model.

Table	PK	FK
User	id	—
Room	id	—
Stay	id	resident_id → User.id, room_id → Room.id
RequestMaintenance	id	user_id → User.id, room_id → Room.id

Participation

We can use extended schema description as hint to figure out participation.

Stay

```
Accross history, User can stay at multiple Room.
Accross history, Room can accommodate multiple User.
```

> Can User never stayed at any Room?

Yes, maybe they just registered. And there is no any Stay record for them.

So User->Stay->Room should be partial participation.

> Can Room never accommodated any User?

Yes, maybe it's a new room and no User ever stayed at.

So Room->Stay->User should be partial participation.

RequestMaintenance

```
Accross history, User can request maintenance for multiple Room.
Accross history, Room can receive request maintenance from multiple User.
```

> Can User never request maintenance for Room?

Yes, maybe User simply doesn't want to report or User never encoutered bad Room.

So User->RequestMaintenance->Room should be partial participation.

> Can Room never receive request maintenance from any User?

Yes, maybe Room is built perfectly flawlesss. Or User stayed at Room never had a chance to live out that Room.

So Room->RequestMaintenance->User should be partial participation.

Relation Model

Both Stay and RequestMaintenance's double direction are partial participation. This also contribute to Relation Model of schema. That is:

> we now know Stay and RequestMaintenance's FK to User and ROOM could be NULL.

Attributes

1. Why Stay carry check_in_at/check_out_at attributes?

I can use them as indicator to whether Room is available or occupied.

If a Room have no correspondant Stay record, that means no User had ever stayed at Room, which means it's empty it's available.

If a Room have correspondant Stay record, but latest record's check_out_at is NULL, that means there is User staying at Room and he hadn't check out, which means Room is occupied.

If a Room have correspondant Stay record, but latest record's check_out_at is not NULL, that means there were indeed some User stayed at Room, but they had checked out. Currently no User staying at Room, which means Room is available.

For some User to stay at Room, he must check in.

> Stay.check_int_at is NOT NULLABLE.

During his stay, Stay's check_out_at is NULL. It's not until User check out then Stay's check_out_at finally have value.

> Stay.check_out_at is NULLABLE.

2. Why RequestMaintenance carry status attribute?

I need to keep track of maintenance progress in 3 phases(submitted/processing/completed).

So I add this attribute, and updating maintenance progress simply means updating this column to next phase. And this is enforeced by Application layer.

CASCADE DEELTE

hy CASCADE DEELTE Stay/RequestMaintenance(room_id) to Room(id) ?

Room and User are solid entities. Stay and RequestMaintenance are relations depend on entities.

So Stay and RequestMaintenance should be deleted if the Room and User they depend on got deleted for data integrity.

Given this view, I should setup `ON DELETE CASCADE` constraint for FK in Stay and RequestMaintenance to PK in User and Room.

But since there is no bussiness need for deleting User. I simply setup constraint for Room.

CREATE VIEW

Why setup RoomDetailed VIEW?

I need to filter Room with status which is no a column in Room schema. It's a derived column but need to join Stay table to calculate it.

The query statement alone to get this extended Room is complex enough. Not to mention I have to combine other WHERE filter.

So it makes sense to store extended Room query as VIEW and reference it like table. Which makes extended Room query with complex filter a lot easier and clearer.

Like this:

```
SELECT  *
FROM    RoomDetailed
WHERE   status = ? AND
        floor = ?
;
```

Conclusion

Room and User are entities, which should be stored as table. Stay and RequestMaintenance are N-M relation, which should also be stored as junction table that use FK referencing User and Room.

Both Stay and RequestMaintenance are bidirectional partial participation, which means their FK pointed to User and ROOM could be NULL.

Add CASCADE DELETE contraint to Stay and RequestMaintenance's room_id FK to Room, so that when Room get deleted related Stay and RequestMaintenance also get deleted to keep data integrity.

Adopting junction table to store relations make query more complex because we need to JOIN multiple tables. Thats's why I introduce RoomDetailed VIEW to simplify query.

AI Disclosure

Tools

- Claude Code

Uasge

1. Generete layout from conceptual UI structure description See: `docs/frames.md`

2. Examine hand written db psuedo code.
3. Examine hand written db actuall code.
4. Detect edge error case, but only mention me without editing and telling answer, I still figured out and patched myself.

Things done by me alone

1. db/feature/view 3 layered archtechture

- db: Database, provide basic sql operation
- feature: API, compose db operations into business logic, validate parameters, raise meaningful error.
- view: UI, provide user interface to access business logic with feature layer.

2. db

- Design db schema
- implement db helper scripts

3. feature

- Catch and handle error
- Compose db operation into business logic